

BUỔI 2: FILES

11/8/2025

Thuật ngữ

Ý nghĩa

Class

Bản thiết kế cho đối tượng

Instance

Đối tượng được tạo từ class

Field / Attribute

Biến dữ liệu trong class hoặc object

Method

Hàm nằm trong class

Parameter

Biến trong định nghĩa hàm

Argument

Giá trị truyền khi gọi hàm

Property

Hàm có hành vi như biến (dùng `@property`)

Namespace

Không gian chứa các tên và giá trị

Constructor (`__init__`)

Hàm đặc biệt khởi tạo đối tượng mới

BASIC FILE READING

- .txt, .json, .csv
- Steps:
 1. **Mở file:** `open(filename, mode)`
 2. **Read content** (`read()`, `readline()` or `readlines()`)
 3. **Đóng file**

```
# Open the file in read mode  
file = open("input.txt", "r")
```

```
# Read the entire content of the file  
content = file.read()  
print(content)
```

```
# Close the file  
file.close()
```

BASIC FILE READING

❖ Xác định đường dẫn

1. Đường dẫn tuyệt đối:

```
f = open("C:/Users/Minh/Desktop/data.txt", "r")
```

Luôn chính xác, khó tái sử dụng khi đổi máy/thư mục.

2. Đường dẫn tương đối

- File cùng thư mục với script: `open("main.py")`
- File nằm trong thư mục con: `open("data/input.txt")`
- File nằm ngoài thư mục hiện tại:

Nếu code nằm trong output/main.py

```
open("../data/input.txt") # lùi 1 cấp ra ngoài
```

3. dùng `pathlib`

```
project/
├── main.py
├── data/
│   └── input.txt
└── output/
    └── result.txt
```

BASIC FILE READING

❖ Xác định đường dẫn

3. dùng `pathlib`

```
from pathlib import Path
```

```
# Thư mục hiện tại của file đang chạy
```

```
base = Path(__file__).parent
```

```
# File trong cùng thư mục
```

```
file1 = base / "data.txt"
```

```
# File trong thư mục con
```

```
file2 = base / "data" / "input.txt"
```

```
# File nằm ngoài thư mục hiện tại
```

```
file3 = base.parent / "config.json"
```

```
project/
```

```
|  
├─ main.py  
├─ data.txt  
├─ data/  
│   └─ input.txt  
└─ config.json  
cấp mới tới)
```

← chứa đoạn code `pathlib`

← file cùng cấp

← file trong thư mục con

← file ở ngoài (main.py lùi 1

BASIC FILE READING

Chế độ	Ý nghĩa	Giải thích
" r "	Read	Mở file để đọc , báo lỗi nếu file không tồn tại (<i>mặc định</i>)
" a "	Append	Mở file để ghi nối thêm vào cuối , tạo mới nếu chưa có
" w "	Write	Mở file để ghi đè nội dung cũ , tạo mới nếu chưa có
" x "	Create	Tạo file mới , báo lỗi nếu file đã tồn tại

BASIC FILE READING

input.txt

Hôm nay
là
chủ nhật

Method	Kiểu dữ liệu trả về	Cách đọc	Ví dụ
<code>read()</code>	str	Toàn bộ file	<code>f.read()</code> 'Hôm nay\nlà\nchủ nhật\n'
<code>readline()</code>	str	1 dòng/lần	<code>f.readline()</code> <code>f.readline()</code> 'Hôm nay\n' 'là\n'
<code>readlines()</code>	list	Toàn bộ (list dòng)	<code>f.readlines()</code> ['Hôm nay\n', 'là\n', 'chủ nhật\n']

READING LINE BY LINE

1. Loop

```
file = open("input.txt", "r")
for line in file:
    print(line.strip()) # .strip() to remove newline characters
file.close()
```

2. readline()

```
file = open("input.txt", "r")
line = file.readline()
while line:
    print(line.strip())
    line = file.readline()
file.close()
```


READING .JSON, CSV, .TXT

```
with open('input.txt', 'r', encoding='utf-8') as f:
    content = f.read()
print(content) # 'Hôm nay\nlà\nchủ nhật\n'
```

```
import json
with open('data.json', 'r', encoding='utf-8') as f:
    data = json.load(f)
print(data["name"]) # An
```

```
import csv
with open('students.csv', newline='', encoding='utf-8') as f:
    reader = csv.reader(f) # có thể DictReader
    next(reader) # Bỏ qua dòng header
    for row in reader:
        print(row[0], row[1])
```

```
# input.txt
Hôm nay
là
chủ nhật
```

```
# data.json
{
    "name": "An",
    "age": 18,
}
```

```
# students.csv
name,age
An,25
Bình,2
```

CREATING A FILE

Mode / Tùy chọn

Mô tả ngắn gọn

"w"	Ghi mới: tạo file nếu chưa có, xóa nội dung cũ nếu đã tồn tại
"a"	Ghi nối: tạo file nếu chưa có, ghi thêm vào cuối nếu đã có
"x"	Tạo độc quyền: chỉ tạo file mới, báo lỗi nếu file đã tồn tại
"b"	Chế độ nhị phân (dùng với ảnh, file .exe, v.v.) – kết hợp với các mode khác như "wb"
"+"	Đọc + ghi cùng lúc, kết hợp với mode khác ("r+", "w+")
encoding=	Chỉ định bảng mã ký tự , ví dụ "utf-8"
newline=	Kiểm soát ký tự xuống dòng , ví dụ "\n"

CREATING A FILE

Mode

Tồn tại file cũ

Tóm tắt ngắn

r+

Bắt buộc, Báo lỗi
`FileNotFoundError`

Đọc & ghi **trên file có sẵn**

w+

Cho phép, **nhưng xóa
sạch**, Tự tạo file mới

Đọc & ghi, **luôn ghi mới lại từ đầu**

CREATING A FILE

❖ Ghi file mới (ghi đè)

```
with open("output.txt", "w") as f:  
    f.write("Xin chào Python!\n")
```

```
import csv  
rows = [{"Name": "Minh", "Age": 25}, {"Name": "Anna", "Age": 30}]  
with open("data.csv", "w", newline="") as f:  
    writer = csv.writer(f)  
    writer.writerows(rows)
```

```
import json  
data = {"name": "Minh", "age": 25, "skills": ["Python", "React"]}  
with open("data.json", "w") as f:  
    json.dump(data, f, indent=4)
```

CREATING A FILE

❖ Ghi nhiều dòng

- **Cách 1:** `writelines()`

ghi từng chuỗi trong list, không tự thêm `\n`.

```
lines = ["First line\n", "Second line\n", "Third\nline\n"]  
with open("file1.txt", "w", encoding="utf-8") as f:  
    f.writelines(lines)
```

- **Cách 2:** `"\n".join() + write()`

Ghép các dòng thành 1 chuỗi rồi ghi một lần (ngắn gọn, sạch hơn).

```
lines = ["Line A", "Line B", "Line C"]  
text = "\n".join(lines) + "\n"  
with open("file2.txt", "w", encoding="utf-8") as f:  
    f.write(text)
```

10.5 LAB: Word frequencies (lists)

Write a program that first reads in the name of an input file and then reads the file using the `csv.reader()` method. The file contains a list of words separated by commas. The program must output the words and their frequencies (the number of times each word appears in the file) **without any duplicates**.

Ex: If the input is:

```
input1.csv
```

and the contents of input1.csv are:

```
hello,cat,man,hey,dog,boy,Hello,man,cat,woman,dog,Cat,hey,boy
```

the output is:

```
hello - 1
cat - 2
man - 2
hey - 2
dog - 2
boy - 2
Hello - 1
woman - 1
Cat - 1
```

Write a program that first reads in the name of an input file and then reads the input file using the **file.readlines()** method. The input file contains an unsorted list of number of seasons followed by the corresponding TV show. Your program should put the contents of the input file into **a dictionary where the number of seasons are the keys, and a Python list of TV shows are the values** (since multiple shows could have the same number of seasons).

Sort the dictionary by key (greatest to least) and output the results to a file named **output_keys.txt**. Separate multiple TV shows associated with the same key with a **semicolon (;)**, ordering by appearance in the input file. Next, **sort the dictionary by values (in reverse alphabetical order)**, and output the results to a file named **output_titles.txt**.

- cấu trúc dữ liệu file?
- (since multiple shows could have the same number of seasons). → kiểu dữ liệu value?
- Nếu key đã tồn tại?
- reverse?